## Extension: optional reading, not graded

### Optional: 4K and GPU scheduling

At the default size first complete the main timing question. Optionally queue a larger image:

Terminal

```
$ sbatch --export=ALL,PROGRAM=trace,ARGS="--compare --res 3840x2160" slurm
    /run.sbatch
```

Read `out/lab10-<jobid>.out`. Does the larger image benefit more? A warp is a group of 32 threads; slower rays can make their neighbours wait.

### Optional: eikonal experiment

The panel samples how quickly $f$ changes, written $|\nabla f|$, at 100 000 points. An exact distance has value 1 where the direction is clear. Run `make trace ARGS="--k 0"`, then $k = 0.2$ and $k = 1$. Optionally tabulate min/mean/max, percentage outside 0.99 to 1.01, and mean steps. These are finite differences and random samples; sharp joins can disagree even at $k = 0$. A field smaller than the true distance still gives safe, shorter steps.

### Optional: topology names

For one connected, closed, orientable manifold surface, $\chi = 2 - 2g$, where genus $g$ counts holes through it. A ball has $g = 0$, our part has $g = 1$. A manifold has two faces per edge and one closed triangle fan per vertex. The judge checks these conditions too. A printed genus of $-1$ means not applicable; it is not a hole count.

## Competition hook (ungraded)

```
1 double extra_cost(const Mesh& m, int v1, int v2, vec3 new_pos);
```

| **What it does** | Adds an optional penalty to discourage a damaging edge collapse. |
|---|---|
| **Gets** | **Meaning** |
| `const Mesh& m` | Current mesh; do not modify it. |
| `int v1` | First endpoint index in m.positions. |
| `int v2` | Second endpoint index in m.positions. |
| `vec3 new_pos` | Proposed replacement position. |
| **Returns** | A double added to the collapse cost. The baseline always returns 0.0 (no extra penalty); it writes nothing. |
| **Example** | For the 2a example with endpoint indices 0 and 1: `extra_cost(m,0,1,(0,0,0))` → 0.0 in the baseline. |

**You can use:** `src/core/mesh.h` gives `Mesh`, `m.positions` (vector of vec3), and `m.tris` (vector of int3 indices with fields x,y,z). For example, `m.positions[v1]` is the first endpoint. The vector helpers and quadric helpers above remain available.

**Steps:**

1. Return 0.0 for the baseline. Later, try an extra cost for a collapse that damages an important feature. This hook is not one of the seven task groups.

The rest is given in `src/core/simplify.cpp`: summing quadrics per vertex, keeping the edges in a list ordered by cost, collapsing the cheapest, updating the neighbours, and two safety checks that refuse a collapse which would tear the mesh or flip a triangle over.

**Check it.** `make check`, `make simplify ARGS=--probe` (the hand example), then

Terminal

```
$ make simplify ARGS="--target 1000"
```

and open `out/simplified.obj` next to `data/part.obj`.

**Going further (competition idea)**

The three candidates are a simple baseline. A better point may lie elsewhere. The given `solve3x3` can find such a point. For this optional idea, `quadric_parts(Q,A,b)` reads the parts of the internal quadric; solve $Av = -b$ with `solve3x3(A,rhs,x)` after negating $b$ into `rhs`. If it returns false, keep the three-candidate answer. Also reject a point farther than $2|v_1 - v_2|$ from the midpoint. Try it after the baseline passes its checks. Task 2a checks the three-candidate rule; for a competition extension, use the independent `judge` verdict instead.

### Optional competition improvements and reference calibration

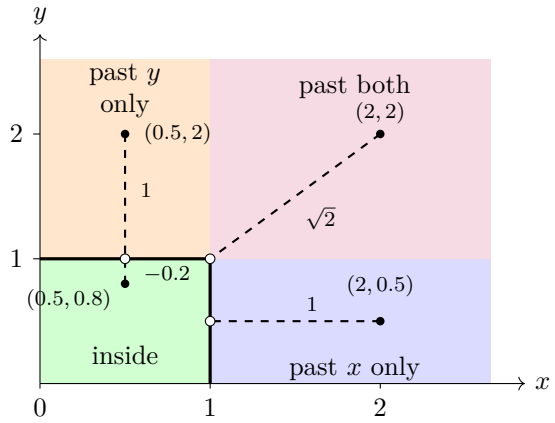
The three-candidate QEM from Part II gives your baseline. Use the calibration below as a reference; measure your own result. To go lower you need your own idea. You may change anything in `student/part2_qem.cu`. The function `extra_cost` is there for this: it is added to the cost of every collapse, and it returns 0 for now. Some directions to think about: what kind of collapse damages the bump or the hole the most? Is the cheapest collapse now always the best one in the end? Could a position outside the three candidates preserve the shape better?

| | | Three candidates | | Optional solve | |
|---|---|---|---|---|---|
| Target | Triangles | Error % | Verdict | Error % | Verdict |
| 200 | 200 | 1.5334 | NOT VALID | 1.3787 | NOT VALID |
| 300 | 300 | 1.4275 | NOT VALID | 0.8921 | NOT VALID |
| 500 | 500 | 0.6298 | NOT VALID | 0.4992 | VALID |
| 750 | 750 | 0.5374 | NOT VALID | 0.3382 | VALID |
| 1000 | 1000 | 0.3158 | VALID | 0.2180 | VALID |
| 1500 | 1500 | 0.3543 | VALID | 0.1028 | VALID |
| 2000 | 2000 | 0.1282 | VALID | 0.0710 | VALID |
| 3000 | 3000 | 0.0507 | VALID | 0.0400 | VALID |

These are reference runs, with the independent judge's 50,000 samples per direction and seed 12345. The smallest valid tested target is 1000 for three candidates and 500 for the optional solve. The rule stays at 0.5%; it leaves room to halve the baseline triangle count. The 500 result is close to the limit (0.4992%), so judge your own saved OBJ. Errors need not decrease at every target: later collapses and the sampled surfaces change.

## Optional: how the box SDF works

For a box with half-sizes $(1, 1)$, look at the top-right quarter below. The other quarters work the same way. Dashed segments join each example to its nearest surface point.




Fold the point with abs, so every coordinate is nonnegative. Set $q = \text{abs}(p) - \text{half_size}$: each component says how far past that wall you are. Outside, keep the positive parts and take their length. Inside, the nearest wall is the largest component of $q$, which is negative. Add the two parts:

$$f(p) = \| \max(q, 0)\| + \min(\max(q_x, q_y), 0).$$

In 3D, include $q_z$ in the inner maximum as well. If you want a challenge, write `sdf_box` yourself and compare with the given one.

## Optional: unsafe collapses

`--unsafe` disables only the link-condition check; flip and zero-area checks remain. Compare these commands. Save the first OBJ before the second run.

```
make simplify ARGS="--target 30"
make simplify ARGS="--target 30 --unsafe"
```

At target 30 the measured safe result has V/E/F=15/45/30, chi=0; unsafe gives 15/44/30, chi=1 and one nonmanifold edge. Both fail the shape-error limit. Equal chi is not sufficient to prove a valid surface.

## Optional: previous 4K timing example

Round 7, RTX 4060 Ti in WSL, 20 host threads (not a B200 measurement):

| Image | CPU single ms | CPU multi ms | GPU ms |
|---|---:|---:|---:|
| 960x540 | 423.827 | 43.926 | 1.387 |
| 3840x2160 | 6307.968 | 647.163 | 3.784 |

Times depend on load. Use your own measurements for the graded default-size comparison.